

(index.html)

UD1 Introducción y entorno de trabajo



UD1 Introducción y entorno de trabajo



Parte I.- Análisis de tecnologías para aplicaciones en dispositivos móviles.



Caso práctico

El uso generalizado en los últimos tiempos de dispositivos móviles como smartPhones o tablets ha dado lugar a una gran demanda de software para este nuevo tipo de hardware. Es por esta razón que **Ada**, socia fundadora de la empresa **BK Programación** y con cierta experiencia en el desarrollo de aplicaciones para dispositivos móviles, ha decidido que su compañía entre también en este nuevo mercado. Se trata de una oportunidad interesante para abrir nuevos negocios tanto para sus clientes actuales como para la captación de otros nuevos.

Para ello necesita formar a sus trabajadores, Juan y María, que acaban de llegar a la oficina.

-¡Buenos días! -saluda Ada-, tengo un nuevo reto para vosotros. Os voy a formar en el desarrollo de aplicaciones para dispositivos móviles, debemos



expandirnos en ese mercado.
-Suenas interesante -responde María-, ¿tienes experiencia en ese campo?
-Sí, he desarrollado aplicaciones en Java para proyectos Java ME y Android Studio.
-Acepto el reto -dice Juan-, ¿cuándo comenzamos?

[Mikhail \(https://www.pexels.com/es-es/foto/empresario-hombre-persona-mujer-6592754/\)](https://www.pexels.com/es-es/foto/empresario-hombre-persona-mujer-6592754/) (Pexels)



[Ministerio de Educación y Formación Profesional.](#)

(Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#)

1.- Primeros conceptos



Caso práctico

Esa misma mañana Ada comienza la formación de María y Juan.

-¿Os habéis dado cuenta del incremento que está habiendo en el uso de dispositivos móviles? -pregunta María-. Quiero que penséis en los diferentes dispositivos móviles que utilizáis en vuestro día a día y por qué los utilizáis, es decir, cuáles son sus ventajas e inconvenientes.

-Haremos la lista de dispositivos móviles -responde María- con características favorables y sus limitaciones.

-Y una vez hecho esa lista -prosigue Ada- debéis plantearos qué debéis tener en cuenta a la hora de desarrollar una aplicación para un dispositivo móvil. En concreto, la tecnología a utilizar.

Juan, viendo que necesitan informarse sobre las características (hardware, software, etc.) de los diferentes dispositivos móviles antes de que Ada continúe su formación, lo ve claro...

-María, nos toca investigar sobre las tecnologías disponibles para el desarrollo de aplicaciones en dispositivos móviles.



Q Mikhail (<https://www.pexels.com/es-es/foto/empresario-hombre-persona-mujer-6592560/>) (Pexels)

El uso generalizado en los últimos tiempos de dispositivos móviles como smartphones, tablets y PDAs ha dado lugar a una gran demanda de software para este nuevo tipo de hardware.

En los próximos apartados, cubriremos de forma general los aspectos más importantes de estas nuevas tecnologías que cada vez están más presentes en el día a día de millones de usuarios, que muchas veces se incorporan a la era digital apoyándose en estos dispositivos sin tan siquiera tener conocimientos previos de informática a nivel de usuario, o prácticamente sin haber utilizado apenas un ordenador. La inmediatez, hiperconectividad y movilidad son aspectos claves para que ya, en la actualidad, la tecnología smartphone y de dispositivo móvil tenga más usuarios en el mundo que las propias tecnologías de PC.



Q Pixabay (CC0)

1.1.- Introducción. ¿Qué es un dispositivo móvil?

La primera pregunta que podemos hacernos es ¿qué entendemos por móvil? Si se nos ocurre investigar sobre ese término, a través de algún buscador de Internet, podremos observar que no hay una respuesta única y que en algunas ocasiones las diferencias pueden ser sustanciales en función de qué es lo que consideremos "móvil". Obviamente esto da lugar a su vez a muchas otras preguntas como por ejemplo: ¿Es móvil... ¿alguna parte del dispositivo? ¿El dispositivo completo? ¿La aplicación que usamos en el dispositivo? ¿Una aplicación cliente? ¿Una aplicación servidor? ¿El usuario del dispositivo? ¿Es "móvil" sinónimo de "portátil"? ¿Es "móvil" sinónimo de "limitado"? ¿Hasta qué punto "móvil" es sinónimo de "autónomo"? ¿Existen diversos grados de "movilidad"? ¿Se pueden clasificar los dispositivos móviles en distintos tipos? ¿Bajo qué criterios? Y así sucesivamente podríamos plantearnos más y más preguntas...

Para evitar este tipo de controversias, en nuestro caso vamos a intentar dar una definición con la que trabajaremos a lo largo del desarrollo del módulo.

¿Qué es un dispositivo móvil?

Se trata de un aparato de pequeño tamaño (normalmente que quepa en un bolsillo) y de poco peso, con pantalla y teclado, con pequeñas capacidades de procesamiento, memoria limitada y conexión (permanente o no) a una red. Este tipo de dispositivos están diseñados para cumplir algún tipo de función específica (realizar llamadas telefónicas, servir como agendas, jugar, navegación GPS, escuchar música, acceso al correo electrónico, navegar por Internet, proporcionar servicio de chat con otros dispositivos móviles, etc.) aunque normalmente pueden llevar a cabo también funciones más generales.

Estos aparatos son cada vez más populares especialmente para aquellos entornos en los que llevar consigo un ordenador convencional (incluso un portátil) no es práctico.

Por otro lado en la definición anterior, en relación a la capacidad de proceso hemos apuntado que tienen "pequeñas capacidades de proceso", lo cual es una verdad a medias, puesto que hoy en día, los dispositivos más avanzados pueden superar a equipos no móviles de gama media-baja de hace apenas 5 ó 6 años. Algo similar ocurre con la memoria, siendo ya frecuente encontrar en el mercado teléfonos móviles con 4 y 6 GB de RAM, e incluso valores más elevados.



Para saber más

Para obtener más información acerca de cuáles pueden ser las características de un dispositivo móvil puedes consultar el siguiente [artículo en la Wikipedia sobre dispositivos móviles](#). 

Clasificación de los dispositivos móviles.

¿Qué tipos de dispositivos móviles existen?

Entre los dispositivos móviles más habituales se encuentran:

- SmartPhones o "teléfonos inteligentes".
- PDA (Personal Digital Assistant - asistentes digitales personales) o PocketPC (PC de bolsillo).
- Handheld, o PCs de mano.
- Internet tables, que se encontrarían entre las PDA y los PC Ultramóviles (pequeños tablet PC).



Q Banco de imágenes de INTEF (CC BY-NC-SA)



Q Jeremy Keith (CC BY)

Según las fuentes que consultes puedes encontrar diversas clasificaciones donde se incluyan unos u otros tipos de dispositivos, como por ejemplo:

- Pagers (o buscapersonas), hoy día ya en desuso.
- Navegadores GPS.
- E-Readers (lectores de libros digitales).
- Pequeñas videoconsolas de mano.
- Cámaras digitales.
- Calculadoras programables.

En nuestro caso, los principales aparatos a los que nos referiremos al hablar de dispositivos móviles serán los smartPhones y las tablets. Por otro lado, según la tecnología vaya avanzando te podrás ir encontrando con nuevos tipos de productos y servicios que irán ampliando las posibilidades de elección. Como siempre sucede en el mundo de la tecnología habrá que estar continuamente al

día de los nuevos avances que van apareciendo para no quedarse atrás.

1.2.- Limitaciones de las tecnologías móviles.

Antes de comenzar a desarrollar software para alguno de estos dispositivos, es necesario ser conscientes de las limitaciones con las que nos podemos encontrar en estos aparatos. ¿Cuáles son las restricciones a las que nos vamos a tener que enfrentar?

Algunas de estas restricciones son:

- Suministro de energía limitado (normalmente dependiente de baterías).
- Procesadores con capacidad de cómputo reducida. Suelen tener una baja frecuencia de reloj por la necesidad de ahorrar energía. En algunos casos, por ejemplo, podrían no disponer de la capacidad de cálculos en punto flotante.
- Poca memoria principal (RAM).
- Almacenamiento de datos persistente reducido (pequeña memoria [flash](#) interna, [tarjetas SD](#), etc.).
- Conexión a algún tipo de red intermitente y con ancho de banda limitado.
- Pantallas de reducidas dimensiones.
- Teclados con funcionalidad muy básica y muy pequeños.



Q Rubaitul Azad (
Licencia de pexels)

Este tipo de restricciones, y algunas otras que dependerán de cada dispositivo en concreto, habrán de ser tenidas muy en cuenta a la hora del análisis y diseño de una aplicación "móvil", pues no podemos pretender, que esa aplicación pueda contener la misma funcionalidad, que la que podemos encontrar habitualmente en un programa que es ejecutado en un ordenador de sobremesa o un portátil.

Por otro lado, no todo va a ser restricciones. También habrá que tener en consideración que esta tecnología va a aportar una serie de ventajas muy importantes: movilidad, poco peso, pequeño tamaño, facilidad para el transporte, conectividad a diversos tipos de redes de comunicaciones (mensajería [SMS](#) y [MMS](#); voz; Internet; [Bluetooth](#); infrarrojos; radiofrecuencia, etc.). Ésas serán las ventajas que podrás explotar en tus aplicaciones.



Para saber más

A través del siguiente vídeo puedes aprender más sobre las características y limitaciones de los dispositivos móviles que han ido solucionándose en los nuevos dispositivos.

<https://www.youtube.com/embed/Fm5GsDUScCA>
(<https://www.youtube.com/embed/Fm5GsDUScCA>)

🔍 [Natalia Arroyo.](https://youtu.be/Fm5GsDUScCA) (<https://youtu.be/Fm5GsDUScCA>) (
Todos los derechos reservados)

 Descripción textual del vídeo 

1.3.- Tecnologías disponibles.

Programar para un dispositivo móvil no va a ser exactamente lo mismo que programar para un ordenador convencional, debido a las limitaciones y características especiales que aquellos aparatos pueden presentar. Hay que conocer qué tipos de tecnologías se pueden encontrar en este ámbito, tanto a nivel de hardware (dispositivos sobre los que se realizarán las aplicaciones) como a nivel de software (sistemas operativos que funcionan sobre esos dispositivos, plataformas de desarrollo disponibles, entornos, APIs, lenguajes de programación, etc.).

Cuando vas a desarrollar una aplicación para un dispositivo móvil, algunas de las primeras preguntas que te puedes hacer son:

- ¿Sobre qué tipos de dispositivos móviles se pueden hacer programas?
¿Sobre qué tipo de hardware se puede programar?
- ¿Qué sistema operativo puede llevar ese hardware?
- ¿Qué plataformas de desarrollo existen para desarrollar sobre ese hardware y ese sistema operativo? ¿con qué lenguajes puedo programar?
¿qué herramientas (compiladores, bibliotecas, entornos, etc.) hay disponibles?

Las respuestas a este tipo de preguntas pueden ser múltiples y muy variadas:

- Respecto al hardware, te puedes encontrar, como has visto ya, principalmente con teléfonos móviles (smartPhones) y tablets (las PDA están actualmente completamente en desuso). Entre los principales fabricantes de teléfonos móviles se encuentran **Samsung, Apple, Huawei, HTC, LG, Motorola, Sony Ericsson, Nokia, Alcatel-Lucent, Xiaomi**, etc. Además en los últimos años fabricantes chinos como **Oppo** y **Vivo** han conseguido hacerse un hueco importante en el mercado, desbancando a muchos de los fabricantes anteriormente mencionados.
- En cuanto a los sistemas operativos, dependiendo del hardware habrá sistemas diseñados para unos u otros dispositivos. Los hay basados en **Microsoft Windows**, en **Linux**, y en **MAC OS X**, así como otros totalmente originales y desarrollados específicamente para estos nuevos tipos de dispositivos. Entre los más populares se encuentran , **Android, iOS, Symbian OS, BlackBerry OS** y **Windows Phone**.
- Si lo que deseas es conocer algo acerca de las plataformas de desarrollo disponibles para cada entorno (hardware y/o sistema operativo), podemos hablar de **Java ME, Windows Mobile SDK, Maemo SDK, Xamarin** o bien de IDEs como **Microsoft Visual Studio, CodeWarrior, Eclipse, Netbeans** y últimamente, con mucha presencia entre programadores profesionales, **Android Studio**.
- Si te refieres a lenguajes de programación, normalmente te encontrarás con lenguajes que son ya viejos conocidos para otras plataformas, como



Q Eduardo Woo (CC BY-SA)

pueden ser las aplicaciones de escritorio para los PCs o las aplicaciones web (**Java, C#, C, Kotlin**, etc.).

En definitiva puedes observar que en este nuevo mundo del desarrollo para dispositivos móviles te encuentras con una problemática similar a la que te puedes enfrentar con los ordenadores convencionales: distintos tipos de hardware, distintas opciones de sistemas operativos dependiendo del hardware que los soporte, diferentes lenguajes de programación, plataformas, API y bibliotecas, entornos de desarrollo, etc.



Para saber más



A través del siguiente [enlace](#) puedes aprender más sobre las PDAs.

Y mediante este otro puedes saber algo más sobre [Xamarin](#).

Q Ryan McVay (http://recursostic.educacion.es/bancoimagenes/ArchivosImágenes/DVD29/CD07/198245_a_1.jpg) (CC BY-NC-SA)

1.3.1.- Hardware

Como has visto en los apartados anteriores, dependiendo de los criterios que se utilicen para clasificar los dispositivos móviles se puede hablar de más o menos tipos. Este curso se va a centrar sobre todo en smartPhones y tablets.

Smartphones

Se puede definir un smartPhone o "teléfono inteligente" como un terminal de telefonía móvil que proporciona unas prestaciones y una funcionalidad mayor que la que podría ofrecer un teléfono móvil normal. Hoy día una buena parte de los teléfonos móviles que se pueden adquirir en el mercado son de este tipo.

Este tipo de terminales se caracteriza por tener instalado un sistema operativo y por tanto la posibilidad de ejecutar aplicaciones desarrolladas bien por el propio fabricante del terminal, bien por el operador de telefonía móvil, o bien por un tercero (empresa de desarrollo de software).

Algunas otras características que suelen tener este tipo de dispositivos son:

- Funcionamiento en multitarea (ejecución concurrente de varios procesos en el sistema operativo).
- Acceso a Internet.
- Conectividad [Wi-Fi](#) , Bluetooth, etc.
- Posibilidad de ampliación de memoria mediante tarjetas externas de memoria (por ejemplo SD).
- Pequeñas pantallas pero de alta resolución y/o con millones de colores.
- Posibilidad de pantallas táctiles o incluso multitáctiles (multitouch).
- Receptor GPS.
- Cámaras digitales integradas. Capacidades fotográficas. Grabación de audio y vídeo.
- Posibilidad de conexión con un ordenador para cargar y descargar información. Normalmente con conexión USB o bien una conexión inalámbrica.
- Sensores (de orientación, de temperatura, de presión, [acelerómetros](#) , [magnetómetros](#) , etc.).
- Receptor de radio [FM](#).



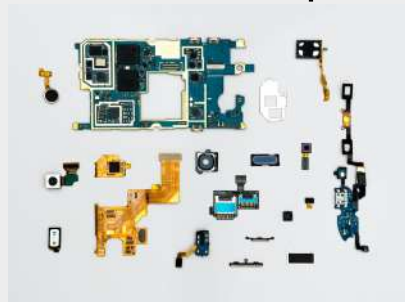
Q [Vojta Kovařík](#) (Licencia [pexels.com](#))

- Emisor de radio FM.
- Posibilidad de instalar y ejecutar aplicaciones sofisticadas:
 - ○ Aplicaciones de asistente personal (gestión de contactos, calendarios, citas, agendas, alarmas, etc.).
 - ○ Gestión del correo electrónico.
 - ○ Gestión del sistema archivos del dispositivo.
 - ○ Microaplicaciones de ofimática (procesador de textos, hoja de cálculo, etc.).
 - ○ Aplicaciones multimedia (reproducción de audio y vídeo en diversos formatos).
 - ○ Aplicaciones de cartografía y navegación.
 - ○ Diccionarios.
 - ○ Pequeñas aplicaciones científicas (matemáticas, física, medicina, etc.).
 - ○ Juegos.
 - ○ Aplicaciones de mensajería instantánea. Chats.

Puedes observar que aunque el dispositivo es un teléfono móvil muchas veces su uso principal no va a ser necesariamente el de un teléfono (hacer y recibir llamadas) sino que podrá estar dedicado a muchos otros usos (hacer fotos, navegar por Internet, reproducir archivos de audio, jugar, gestionar la agenda personal, consultar un mapa, usar un diccionario, escuchar la radio, ver una película, trazar una ruta para el navegador por satélite, etc.).

Dentro de muy poco tiempo es probable que se deje de emplear el término smartPhone o "móvil inteligente" pues todos los terminales móviles serán de este tipo y se hablará simplemente de "teléfonos móviles". De hecho en la práctica ya ocurre hoy día, pues cuando nos acercamos a algún catálogo de productos la inmensa mayoría de terminales que se nos ofrecen son de este tipo.

El interior de un smartphone



Q [Dan Cristian Pădureț](#) (Licencia [Pexels.com](#))

Se encontrarían el procesador principal, procesadores digitales de señal (DSP), procesadores de imagen y de audio, módem, memorias caché, co-

Si se te ocurriera abrir un teléfono móvil actual te encontraríamos una placa de circuito impreso con una serie de dispositivos electrónicos integrados o insertos en ella. En cierto modo podría recordar un poco a la placa base de un PC. Entre los dispositivos que podrías observar se

dec de audio y vídeo, etc. A este conjunto de dispositivos integrados en la placa se le suele llamar chipset (lo cual te debería recordar al mundo del hardware de los PCs y a los conceptos estudiados en el módulo **Sistemas Informáticos**).

La principal arquitectura de microprocesadores utilizada para telefonía móvil es la diseñada por la empresa **ARM** ("Advanced RISC Machines", hoy día ARM Holdings), teniendo actualmente más del 90% de la cuota de mercado no ya sólo para teléfonos móviles, sino en general para todo tipo de pequeños dispositivos electrónicos que necesitan un microprocesador. Algunas de las familias de micros **ARM** más conocidas son las **ARM7**, **ARM9**, **ARM11** y **Cortex**. Hoy día muchas otras empresas diseñan microprocesadores basados en las patentes de la arquitectura de los procesadores **ARM** como por ejemplo **DEC**, **IBM**, **Texas Instrument**, **Samsung**, **Intel**, **Atmel**, **Alcatel-Lucent**, **Apple**, **Qualcomm**, **LG**, **Nintendo**, **Philips**, **Oki**, **Yamaha**, etc. Uno de los más conocidos es por ejemplo el **Intel XScale**.

Algunos ejemplos de productos que incorporan uno o varios micros ARM son: la mayoría de teléfonos móviles (Samsung, Apple, etc.), el iPod de **Apple**, las consolas **Gameboy** y **Nintendo DS**, routers, navegadores GPS, cámaras digitales, etc.

Respecto al chipset, existen diversas posibilidades de arquitectura, según sea el fabricante que las produce. Algunas de las arquitecturas más conocidas son **MSM** (de la empresa **Qualcomm**), **OMAP** (Open Multimedia Application Platform, de la empresa Texas Instrument), y las de la empresa **Marvell**.

Muchos teléfonos de **Nokia** utilizan la arquitectura **OMAP** de **Texas Instrument** (por ejemplo el **Nokia N810 WiMax** Edition que incorpora un chipset **OMAP 2420** con un procesador **ARM1136**), mientras que algunas **BlackBerries** hacen uso de la **MSM** de **Qualcomm** (por ejemplo la **BlackBerry Storm**, con una arquitectura **MSM7600**) o de arquitecturas **Marvell** (por ejemplo la **BlackBerry Bold**, con un chipset **Marvell Travor PXA930**). Algunos modelos de **Samsung** también se han decantado por **Marvell** (por ejemplo el **Samsung Omnia**, que incorpora un chipset **Marvell** a 624 Mhz).

2.- Sistemas operativos.



Caso práctico

María y Juan ya tienen claro qué es un dispositivo móvil y las diferentes tecnologías disponibles. Ada continúa la formación de sus empleados...

-Como habéis descubierto, cada hardware maneja un sistema operativo diferente.

-Sí -responde María-, hay que tener en cuenta el hardware del dispositivo móvil.

-Y el hardware depende del fabricante - añade Juan.

-Exacto -confirma Ada-, por lo ahora os toca conocer las principales características de los sistemas operativos más utilizados actualmente en el mercado.



Q Mikhail (<https://www.pexels.com/es-es/foto/empresario-hombre-persona-mujer-6592670/>). (Pexels)

Los sistemas operativos más habituales que te puedes encontrar en un dispositivo móvil son:

- **Android**. Desarrollado inicialmente por **Google** y basado en el núcleo de **Linux**. El primer fabricante de móviles que lo incorporó fue **HTC**.
- **iOS**. Desarrollado por **Apple** para el **iPhone** y usado más tarde también para el **iPod Touch** y el **iPad**.
- **Windows Phone** (anteriormente llamado **Windows Mobile** y posteriormente derivado en **Windows 10 Mobile**) (discontinuado). Desarrollado por **Microsoft** tanto para smartPhones como para otros dispositivos móviles (por ejemplo PDA). Algunos fabricantes de teléfonos móviles que incorporaban este sistema operativo son Samsung, LG o HTC.
- **Blackberry OS** (discontinuado). Desarrollado por **Research in Motion (RIM)** para sus dispositivos **Blackberry**.

Según la fuente [Global stat \(statcounter.com\)](https://www.statcounter.com), en diciembre de 2021, la cuota de mercado de sistemas operativos para móviles se encuentra distribuida de la siguiente forma a nivel mundial:

Sistema Operativo	Android	iOS	Samsung	KaiOS	Unknown	Nokia Unknown	Wir
Cuota de Mercado	70.01%	29.24%	0.43%	0.13%	0.12%	0.002%	0.

Según esa misma fuente y para la misma fecha, la cuota de mercado de sistemas operativos para tabletas se encuentra distribuida de la siguiente forma

a nivel mundial:

Sistema Operativo	iOS	Android	Windows	Linux
Cuota de Mercado	55.66%	44.25%	0.04%	0.02%

Si además trasladamos estas estadísticas a nivel nacional, se puede decir que en España la cuota de mercado de Android en smartphones supera el 78% del mercado.

Vamos a hacer un rápido repaso de algunos de los sistemas operativos más conocidos: **Android**, **iOS** e **Windows Phone**.

2.1.- Android.



android

Q [Android, Google](#) (
[Wikimedia Commons](#))

Android fue inicialmente desarrollado por **Android Inc.**, hoy día parte de la compañía **Google**. Está basado en una versión modificada del [kernel](#) de **Linux**.

El primer fabricante que incorporó **Android** en sus dispositivos fue **HTC** con su terminal **HTC Dream** (comercializado también como **T-Mobile G1** y popularmente conocido con los nombres de **Google Phone** o **GPhone**) en 2008.

Es uno de los más "jóvenes" dentro del grupo de sistemas operativos para dispositivos móviles y se ha hecho un notable hueco alcanzando más de las dos terceras partes de la cuota de mercado actual. Existen miles de aplicaciones que funcionan sobre **Android**, con un crecimiento cada vez mayor.

Hoy día podemos encontrar multitud de fabricantes que incorporan **Android** en sus terminales: **Samsung** (p.e., series **Galaxy** y **Nexus**), **Xiaomi**, **Huawei** (en las últimas versiones no se incorpora por políticas de Google), **Oppo**, **Vivo**, **LG**, **Sony Ericsson** (p.e., series **Xperia**), **HTC** (p.e., Series **A**), **Acer** (p.e. series **beTouch**), **Motorola** (p.e., **Droid X**), **Garmin**, **Dell**, **Alcatel**, etc.

A diciembre de 2021, la última versión, de Android es 12.0. Hasta las versión Android tenía un simpático criterio de asignación de nombres para las diferentes versiones que se correspondían con postres y que avanzaban la versión según avanzaba alfabéticamente la letra del postre.

Para desarrollar aplicaciones sobre **Android** se necesita el kit de desarrollo de software para Android (**Android SDK**), proporcionado gratuitamente por **Google**. Este paquete incluye todo lo necesario para construir aplicaciones sobre el entorno **Android** (depurador, bibliotecas, emulador, documentación, etc.). El lenguaje de programación que se utiliza es **Java** (y por tanto es necesario el **JDK** de **Oracle** para poder compilar programas **Java**).

El IDE oficial ha sido, durante varios años, **Eclipse** (a partir de la versión 3.2) junto con el plugin **ADT** (**Android Development Tools** - herramientas de desarrollo para **Android**). Sin embargo, las desavenencias entre **Oracle** (propietaria de **Java**) y **Google** (propietaria de **Android**) en los últimos años han provocado que **Android** abandonara la recomendación de uso de la versión ME de **Eclipse** para dispositivos móviles y haya desarrollado su propio IDE partiendo del IDE **IntelliJ IDEA** de la compañía **JetBrains**, convirtiéndose, a partir de finales del año 2013, en el nuevo IDE oficial para el desarrollo de aplicaciones para **Android**.

Nombre código	Número de versión	Fecha de lanzamiento
Apple Pie TM	9.0	23 de septiembre de 2019
Banana Cream TM	9.1	9 de febrero de 2020
Chocolate	9.0	25 de abril de 2020
Donut	1.6	19 de septiembre de 2009
Eclair	2.0 - 2.1	26 de octubre de 2009
Froyo	2.2 - 2.2.2	20 de mayo de 2010
Gingerbread	2.3 - 2.3.7	6 de diciembre de 2010
Honeycomb TM	3.0 - 3.2.1	22 de febrero de 2011
Ice Cream Sandwich	4.0 - 4.0.3	19 de octubre de 2011
Jelly Bean	4.1 - 4.2.2	9 de julio de 2012
KitKat	4.4 - 4.4.4	17 de octubre de 2013
Lollipop	5.0 - 5.1.1	12 de noviembre de 2014
Marshmallow	6.0 - 6.0.1	5 de octubre de 2015
Nougat	7.0 - 7.1.2	15 de junio de 2016
Oreo	8.0 - 8.1	21 de agosto de 2017
Pie	9.0	6 de agosto de 2018
Q	10.0	5 de septiembre de 2019
R	11.0	4 de septiembre de 2020
S	12.0	4 de octubre de 2021

Q [Wikipedia](#). Captura de
pantalla con las versiones
de Android (Dominio público)





Para saber más sobre Android se pueden consultar los [artículos de la Wikipedia sobre este sistema operativo](#), así como los documentos a los que hace referencia:

2.2.- iOS.

Se trata del sistema operativo desarrollado por **Apple** originalmente para su **iPhone**, aunque hoy día también es utilizado por otros dispositivos de la empresa.

Apple sólo permite que este sistema operativo funcione sobre hardware **Apple**. Es un sistema operativo derivado del **Mac OS X**, también de **Apple**.



Q LoboStudioHamburg (
Pixabay License)

En enero de 2022, según www.apple.com, la tienda de aplicaciones de **Apple** (servicio de descarga de aplicaciones), llamada **App Store**, dispone ya de 1,8 millones de aplicaciones para dispositivos con iOS. En la misma fecha, la cuota de mercado de Apple en los dispositivos móviles es del 29,21% según gs.statcounter.com.

La última versión de **iOS** en octubre de 2021 es la versión de 15.0.2. Como se ha comentado antes, los únicos dispositivos que incorporan este sistema operativo son los distintos modelos de dispositivos fabricados por la propia compañía **Apple**. Es decir, los **iPhone**, **iPad**, **iPod Touch**.

En 2015, **Apple** lanza su **Apple Watch**. Un reloj inteligente (**smartwatch**) que requiere un **iPhone** asociado para funcionar. Usa el sistema operativo **watchOS** basado en **iOS**.

Para desarrollar aplicaciones sobre **iOS**, las aplicaciones deben ser compiladas específicamente para este sistema operativo basado en la [arquitectura ARM](#). Para ello puede utilizarse el kit de desarrollo **iPhone SDK**. El lenguaje de programación principal para este conjunto de herramientas es el **Objective-C**. Para poder utilizar este kit de desarrollo es necesario un ordenador **MAC** con un sistema operativo **MAC OS X Leopard** o superior. A mediados de 2011 ni el entorno **.NET** y ni **Adobe Flash** eran soportados por **iOS**, aunque **Adobe Flash** es utilizable a través de aplicaciones de terceros. Lo mismo sucede con **Java**, aunque se han producido algunos intentos de máquinas virtuales de **Java** (basadas en **JAVA ME**) para **iOS**, que rozan la legalidad a la que están sometidas las restrictivas licencias de **Apple**. Por supuesto, siempre habrá comunidades de usuarios y desarrolladores que intentarán saltarse estas restricciones (para más información al respecto puedes consultar el término **iOS Jailbreaking** en la web).

El **iOS SDK** se puede descargar gratuitamente, aunque es necesario registrarse en el [Programa del desarrollador de Apple](#) para poder publicar el software creado, lo cual requiere la aprobación de la compañía **Apple** y un pago por ese servicio, que proporcionará al programador las claves firmadas que le permitirán publicar en la **App Store**.



Para saber más

Para saber más sobre **iOS** se pueden consultar los artículos de la [Wikipedia sobre este sistema operativo](#),, así como los documentos externos a los que hace referencia.

2.3.- Windows Phone.

Es el sistema operativo desarrollado por **Microsoft** para **smartPhones** y otros dispositivos móviles. Sustituye a su antecesor **Windows Mobile**. Las primeras versiones de sistemas operativos para dispositivos móviles desarrolladas por **Microsoft** eran conocidas como **Windows CE (Compact Embeded)**, y de hecho la familia de sistemas operativos de **Microsoft** para sistemas empujados en general (**smartPhones**, **PDA** y otros pequeños dispositivos) sigue llamándose **Windows CE**, así como el núcleo del sistema operativo.

Algunos de los primeros **Windows CE** podían encontrarse en dispositivos como las **PDA iPAQ** de **Compaq** (hoy día parte de HP), las cuales surgieron como competencia directa de las **PDA** de **Palm** (las **Palm Pilot**) durante la segunda mitad de la década de los noventa. Las **iPAQ** con **Windows CE** ofrecían un entorno más parecido al **Windows** de los ordenadores de escritorio frente al aspecto más sobrio de la interfaz de usuario de **Palm OS**.

Microsoft anunció en enero de 2015 que daría de baja a **Windows Phone**, para enfocarse en un único sistema más versátil denominado **Windows 10 Mobile**, disponible para todo tipo de plataformas (teléfonos inteligentes, tabletas y computadoras). Así, **Microsoft** lanzó su sistema operativo para ordenadores **Windows 10** en julio de 2015, y finalmente, marzo de 2016, lanzó su equivalente para dispositivos móviles **Windows 10 Mobile**. Actualmente es un sistema operativo discontinuado.



ArtificialOg (Pixabay License)

La última versión de **Windows Phone** fue la 8.1 (lanzada en marzo de 2015), desarrollada propiamente para **smartPhones** y **PDA**. Algunas de las versiones anteriores han sido: **Windows Phone 8**, **Windows Phone 7.x**, **Windows Mobile 6.x**, **Windows Mobile 5.x**, **Windows Mobile 2003** o **Pocket PC 2002**. Todas ellas basadas en la plataforma **Windows CE**.

Entre los fabricantes que incluían **Windows Phone** en sus dispositivos se encuentran **HTC** (series S y T), **Samsung** (serie Omnia), **LG** (Optimus 7 y Pacific) y **Sony Ericsson** (Xperia X7) y **Nokia**.

Para desarrollar aplicaciones sobre **Windows Phone** pueden utilizarse las tecnologías **Silverlight** y **XNA** de **Microsoft**, así como el entorno de desarrollo **Visual Studio**.



Para saber más

Para saber más sobre [Windows Phone](#) y [Windows 10 Mobile](#) se pueden consultar los artículos de la Wikipedia sobre este sistema operativo, así como los documentos externos a los que hace referencia.

3.- Plataformas de desarrollo y lenguajes de programación.



Caso práctico

María y Juan ya conocen los sistemas operativos con los que van a trabajar y para los que van a desarrollar las aplicaciones de los diferentes dispositivos móviles. A María le surge una duda...

-Para los desarrollos de software que estamos acostumbrados a realizar, siempre nos ayudamos de algún editor que nos simplifica el trabajo -reflexiona María-, entiendo que tendremos también alguna ayuda en el desarrollo de software para dispositivos móviles.

-Así es -confirma Ada-, en este caso tenemos plataformas de desarrollo.

-¡Claro! -exclama Juan-, y cada plataforma trabajará con un lenguaje de programación.

-En efecto. Os los voy a mostrar...



Q Mikhail (<https://www.pexels.com/es-es/foto/empresario-hombre-persona-mujer-6592753/>). (Pexels)

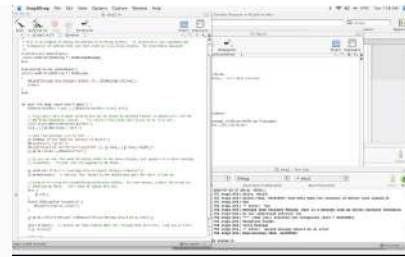
Para poder desarrollar aplicaciones para alguno de los anteriores sistemas operativos es necesario, disponer de alguna plataforma de desarrollo que permita generar código ejecutable sobre esos sistemas, o bien sobre alguna máquina virtual, o algún intérprete que esté instalado en el dispositivo y que sea soportado por el sistema operativo.

Dependiendo de la versatilidad de la plataforma de desarrollo, la aplicación podrá ser, más o menos portable a otros sistemas operativos y/o dispositivos. Por ejemplo, si realizamos programas en **Objective-C** utilizando el **SDK** de **Apple**, lo más probable es que nuestra aplicación pueda ser únicamente ejecutada en un dispositivo **Apple** (**iPhone** por ejemplo) sobre el que esté implantado **iOS**. Si, por el contrario, desarrollamos un programa en **Java** usando el **Java ME** de **Oracle**, y generamos una aplicación tipo **MIDlet** , ésta podrá ser ejecutada en cualquier dispositivo que disponga de una pequeña máquina virtual de **Java** (**KVM**). Se trata nuevamente de la misma problemática que podemos encontrar en el mundo de los ordenadores convencionales.

Entre las plataformas de desarrollo para móviles más populares se encuentran normalmente las que los propios autores de los sistemas operativos ofrecen para trabajar sobre su plataforma. De hecho ya hemos hablado algo sobre algunas de ellas al describir los sistemas operativos. Hacemos un repaso rápido de las más conocidas:

- **Android.** Google proporciona también de manera gratuita el **Android SDK**

para programar aplicaciones en **Java** sobre su sistema operativo. El **IDE** más utilizado en el pasado fue **Eclipse** con el plugin **ADT** (Android Development Tools) y últimamente se impone **Android Studio**. También es posible programar en otros lenguajes como **C** o **C++** gracias al uso del



Q [Ron Bieber](#) (CC BY-NC-ND)

Android NDK (Native Development Kit) que permite generar código nativo (native code), frente al código gestionado (managed code), que se ejecuta sobre una máquina virtual, como es el caso de **Java** o **C#**. Por otro lado, no se debe perder de vista el deliberado impulso que está haciendo **Google** para que **Android** sea programado en lenguaje **Kotlin**.

- **iOS**. El **SDK** también se puede descargar gratis, pero para poder comercializar el software a través de su tienda de aplicaciones hay que registrarse en el programa de desarrollo del **iPhone**, lo cual no es gratuito. El lenguaje de programación en este caso es **Objective-C**.



Q [LearnTek](#) (CC0)

- **Java ME**: Este caso sería una excepción respecto a los demás, pues no se trata de una serie de herramientas (bibliotecas, compiladores, IDEs, etc.) asociadas a un sistema operativo, sino de un subconjunto de la plataforma **Java** orientada para el desarrollo sobre dispositivos móviles. Se programaría en lenguaje **Java** y sería necesario que hubiera

instalada una máquina virtual en el sistema operativo del dispositivo sobre el que se deseara ejecutar la aplicación desarrollada como sucede por ejemplo en **Symbian** (que se puede programar en modo nativo o bien con **Java ME**).

- **Windows Phone** (discontinuado). Para desarrollar aplicaciones sobre este sistema operativo pueden utilizarse las tecnologías **Silverlight**, **XNA** y **.NET Compact Framework** de **Microsoft**. Las herramientas **Visual Studio 2010 Express** y **Expression Blend** para **Windows Phone** son ofrecidas gratuitamente por **Microsoft**. El lenguaje más habitual para el desarrollo ha sido **C#**, aunque la idea es que se pueda utilizar también otros lenguajes de la plataforma **.NET** como **Visual Basic .NET**.
-



Para saber más

Para saber más sobre el lenguaje Kotlin:

- [Artículo de xatakandroid.com sobre kotlin](https://www.xatakandroid.com/programacion-android/kotlin-ya-es-un-lenguaje-oficial-en-android-que-implicaciones-tiene-y-por-que-es-tan-importante)  (<https://www.xatakandroid.com/programacion-android/kotlin-ya-es-un-lenguaje-oficial-en-android-que-implicaciones-tiene-y-por-que-es-tan-importante>)
- [Artículo de apiumhub.com sobre kotlin](https://apiumhub.com/es/tech-blog-barcelona/por-que-kotlin/)  (<https://apiumhub.com/es/tech-blog-barcelona/por-que-kotlin/>)
- [Artículo de paradigmadigital.com sobre kotlin](https://www.paradigmadigital.com/dev/kotlin-otra-moda-mas-spoiler-no/)  (<https://www.paradigmadigital.com/dev/kotlin-otra-moda-mas-spoiler-no/>)

3.1.- Elección de una alternativa.

Dado que durante este Ciclo Formativo, es probable que hayas programado ya en **Java** (módulo de Programación), las alternativas más propicias son **Java ME** y **Android Studio** sobre **Android**. En cualquier caso, hemos de ser siempre conscientes de que no se trata más que de una de las posibles alternativas disponibles en el mercado y que más adelante (en futuros proyectos dentro de la empresa) es probable que quizá tengas que trabajar con otros lenguajes (por ejemplo **C#, C++, Python, Objective-C, Kotlin,...**) y para otras plataformas.

Lo importante es adquirir unos conocimientos generales sobre este nuevo tipo de entornos, sobre algunas bibliotecas concretas para estos dispositivos (por ejemplo los paquetes **javax** en el caso de **Java ME**) y comenzar a desarrollar algunos ejemplos de aplicaciones para ir desarrollando una nueva filosofía de trabajo.

En nuestro caso Java puede ser una buena elección dado que es el lenguaje con el que aprendiste a programar en el módulo de Programación y por tanto no tendrás que aprender un nuevo lenguaje y podrías dedicarte de lleno al aprendizaje de las características específicas para el desarrollo de aplicaciones para dispositivos móviles. Por otro lado, un lenguaje multiplataforma como **Java** es también muy adecuado para un ciclo con un nombre como "Desarrollo de Aplicaciones Multiplataforma". Pero en cualquier caso no debes olvidar que se trata de una de las muchas opciones con las que te puedes encontrar en el mercado, y que estas alternativas irán apareciendo y desapareciendo constantemente en función del éxito que vayan obteniendo.

Respecto a la plataforma, debemos elegir entre aquellas que estén relacionadas con **Java** (**Java ME** y **Android Studio**). Concretamente, nos decantaremos por **Android Studio**, que es la que ha experimentado en los últimos años una progresión mayor.



Q Jaumemonasterio (CC BY-SA)

Resumiendo, para facilitarte el aprendizaje vamos a trabajar con:

- **Plataforma Android.**
- **Lenguaje Java y XML.**
- **IDE Android Studio.**
- Sistema operativo que permita una máquina virtual **Java** para la emulación, aunque normalmente usaremos un emulador proporcionado por el propio IDE.
- Hardware (smartPhones, tablets, etc.) que soporte un sistema operativo del tipo anterior.

Como suele suceder en el mundo real, normalmente no podrás elegir una opción u otra en función de tus preferencias personales, sino más bien dependiendo de cuáles sean las necesidades de tu cliente. Esa es una buena razón para intentar estar siempre al día.

3.2.- La plataforma Android.

Los componentes principales del sistema operativo de **Android**:

- **Aplicaciones:** las aplicaciones base incluyen un cliente de correo electrónico, programa de SMS, calendario, mapas, navegador, contactos y otros. Todas las aplicaciones están escritas en lenguaje de programación Java.
- **Marco de trabajo de aplicaciones:** los desarrolladores tienen acceso completo a los mismos API del entorno de trabajo usados por las aplicaciones base. La arquitectura está diseñada para simplificar la reutilización de componentes; cualquier aplicación puede publicar sus capacidades y cualquier otra aplicación puede luego hacer uso de esas capacidades (sujeto a reglas de seguridad del framework). Este mismo mecanismo permite que los componentes sean reemplazados por el usuario.
- **Bibliotecas:** Android incluye un conjunto de bibliotecas de C/C++ usadas por varios componentes del sistema. Estas características se exponen a los desarrolladores a través del marco de trabajo de aplicaciones de Android. Algunas son: System C library (implementación biblioteca C estándar), bibliotecas de medios, bibliotecas de gráficos, 3D y SQLite, entre otras.
- **Runtime de Android:** Android incluye un set de bibliotecas base que proporcionan la mayor parte de las funciones disponibles en las bibliotecas base del lenguaje Java. Cada aplicación Android corre su propio proceso, con su propia instancia de la máquina virtual Dalvik. Dalvik ha sido escrito de forma que un dispositivo puede correr múltiples máquinas virtuales de forma eficiente. Dalvik ejecutaba hasta la versión 5.0 archivos en el formato de ejecutable Dalvik (.dex), el cual está optimizado para memoria mínima. La Máquina Virtual está basada en registros y corre clases compiladas por el compilador de Java que han sido transformadas al formato.dex por la herramienta incluida dx. Desde la versión 5.0 utiliza el ART, que compila totalmente al momento de instalación de la aplicación.
- **Núcleo Linux:** Android depende de Linux para los servicios base del sistema como seguridad, gestión de memoria, gestión de procesos, pila de red y modelo de controladores. El núcleo también actúa como una capa de abstracción entre el hardware y el resto de la pila de software.



Q Torsten Dettlaff (pexels.com)

3.3.- El entorno de ejecución.



Debes conocer

A partir de ahora haremos muchas referencias durante todo el curso a la documentación oficial proporcionada por Google sobre el desarrollo de Android (developer.android.com). De hecho se puede decir que es la principal guía para elaborar los contenidos relativos a Android de este curso.

Si consultamos la página oficial de desarrollo de Android (developer.android.com) podemos observar la siguiente estructura de capas o niveles.

Analicemos las diferentes capas empezando por abajo:

A. Kernel de Linux: La base de la plataforma Android es el kernel de Linux. Por ejemplo,



Q Google Developers (

Apache 2.0)

el tiempo de ejecución de Android (ART, acrónimo de Android Run Time) se basa en el kernel de Linux para funcionalidades subyacentes, como la generación de subprocesos y la administración de memoria de bajo nivel. El uso del kernel de Linux permite que Android aproveche funciones de seguridad claves y, al mismo tiempo, permite a los fabricantes de dispositivos desarrollar controladores de hardware para un kernel conocido.

B. Capa de abstracción de hardware (HAL). La capa de abstracción de hardware (HAL) brinda interfaces estándares que exponen las capacidades de hardware del dispositivo al framework de la Java API de nivel más alto. La HAL consiste en varios módulos de biblioteca y cada uno de estos implementa una interfaz para un tipo específico de componente de hardware, como el módulo de la cámara o de bluetooth. Cuando el framework de una API realiza una llamada para acceder a hardware del dispositivo, el sistema Android carga el módulo de biblioteca para el componente de hardware en cuestión.

C. Tiempo de ejecución de Android (ART): Para los dispositivos con Android 5.0 (nivel de API 21) o versiones posteriores, cada app ejecuta sus propios procesos con sus propias instancias del tiempo de ejecución de Android (ART). El ART está escrito para ejecutar varias máquinas virtuales en dispositivos de memoria baja ejecutando

archivos DEX, un formato de código de bytes diseñado especialmente para Android y optimizado para ocupar un espacio de memoria mínimo. Crea cadenas de herramientas, como [Jack](#), y compila fuentes de Java en código de bytes DEX que se pueden ejecutar en la plataforma Android. Estas son algunas de las funciones principales del ART:

- Compilación ahead-of-time (AOT) y just-in-time (JIT)
- Recolección de elementos no usados (GC) optimizada
- Mejor compatibilidad con la depuración, como un generador de perfiles de muestras dedicado, excepciones de diagnóstico detalladas e informes de fallos, y la capacidad de establecer puntos de control para controlar campos específicos.

Antes de Android 5.0 (nivel de API 21), Dalvik era el tiempo de ejecución del sistema operativo. Por tanto ART es una evolución de Dalvik mejorando su rendimiento, en cuanto a velocidad de ejecución y tiempo que tarda en abrir una aplicación. Si tu app se ejecuta bien en el ART, también debe funcionar en Dalvik, pero es posible que no suceda lo contrario.

En Android también se incluye un conjunto de bibliotecas de tiempo de ejecución centrales que proporcionan la mayor parte de la funcionalidad del lenguaje de programación Java; se incluyen algunas [funciones del lenguaje Java 8](#), que el framework de la Java API usa.

D. Bibliotecas C/C++ nativas: Muchos componentes y servicios centrales del sistema Android, como el ART y la HAL, se basan en código nativo que requiere bibliotecas nativas escritas en C y C++. La plataforma Android proporciona la API del framework de Java para exponer la funcionalidad de algunas de estas bibliotecas nativas a las apps. Por ejemplo, puedes acceder a la [guía](#) de [OpenGL ES](#) a través de la [Java OpenGL API](#) del framework de Android para agregar a tu app compatibilidad con los dibujos y la manipulación de gráficos 2D y 3D.

Si desarrollas una app que requiere C o C++, puedes usar el [NDK de Android](#) para acceder a algunas de estas [bibliotecas de plataformas nativas](#) directamente desde tu código nativo.

E. Framework de la API de Java: Todo el conjunto de funciones del SO Android está disponible mediante API escritas en el lenguaje Java. Estas API son los cimientos que necesitas para crear apps de Android simplificando la reutilización de componentes del sistema y servicios centrales y modulares, como los siguientes:

- Un [sistema de vista](#) enriquecido y extensible que puedes usar para compilar la IU de una app; se incluyen listas, cuadrículas, cuadros de texto, botones e incluso un navegador web integrable.
- Un [administrador de recursos](#) que te brinda acceso a recursos sin código, como strings localizadas, gráficos y archivos de diseño.
- Un [administrador de notificaciones](#) que permite que todas las apps muestren alertas personalizadas en la barra de estado.

- Un **administrador de actividad**  que administra el ciclo de vida de las apps y proporciona una **pila de retroceso de navegación**  común.
- **Proveedores de contenido**  que permiten que las apps accedan a datos desde otras apps, como la app de Contactos, o compartan sus propios datos.

Los desarrolladores tienen acceso total a las mismas **API del framework**  que usan las apps del sistema Android.

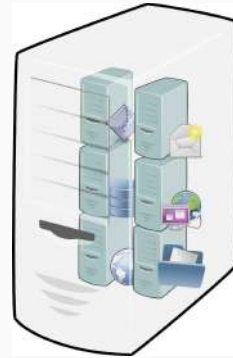
F. Apps del sistema: En Android se incluye un conjunto de apps centrales para correo electrónico, mensajería SMS, calendarios, navegación en Internet y contactos, entre otros elementos. Las apps incluidas en la plataforma no tienen un estado especial entre las apps que el usuario elige instalar; por ello, una app externa se puede convertir en el navegador web, el sistema de mensajería SMS o, incluso, el teclado predeterminado del usuario (existen algunas excepciones, como la app Settings del sistema).

Las apps del sistema funcionan como apps para los usuarios y brindan capacidades claves a las cuales los desarrolladores pueden acceder desde sus propias apps. Por ejemplo, si en tu app se intenta entregar un mensaje SMS, no es necesario que compiles esa funcionalidad tú mismo; como alternativa, puedes invocar la app de SMS que ya está instalada para entregar un mensaje al receptor que especifiques.

3.4.- Maquinas virtuales.

Una máquina virtual es una aplicación software que emula el comportamiento de otra máquina (por ejemplo la emulación de un ordenador "virtual" dentro de un ordenador "real").

En el caso de una máquina virtual de **Java** (JVM – Java Virtual Machine), si recordamos lo visto en el módulo de **Programación**, se trata de un programa encargado de interpretar código intermedio (bytecode en el caso de Java) de los programas precompilados (en lenguaje Java) a código máquina ejecutable por el hardware (instrucciones máquina), realizar las llamadas al sistema operativo necesarias, velar por la seguridad e integridad del código ejecutado, etc.



Q [OpenClipart-Vectors](#) (
[Pixabay License](#))

De esta manera, la **JVM** proporciona al programa compilado en **Java** independencia de la plataforma hardware y del sistema operativo que corra sobre él.

Las **JVM** clásicas para la plataforma **Java SE** son demasiado pesadas (necesidades de memoria, requerimientos de cómputo, pantalla, etc.) para dispositivos pequeños. Imagina por ejemplo la limitada pantalla de un teléfono móvil. Es fácil suponer que ese tipo de pantallas serían incapaces de soportar toda la funcionalidad proporcionada por la **AWT**, que es la principal interfaz gráfica de usuario que tiene **Java**. Es por esta razón que la plataforma **Java ME** ha propuesto otras máquinas virtuales bastante más ligeras.

Por otro lado, una única plataforma de Java podría no encajar adecuadamente con todos los modelos de dispositivos posibles. Es por ello por lo que **Java ME** introdujo los conceptos de configuración y de perfil.

Las dos máquinas virtuales disponibles en **Java ME** son la **KVM (K Virtual Machine)**, se le ha dado ese nombre porque sólo ocuparía unos pocos Kilobytes de memoria) y la **CVM (Compact Virtual Machine)**, algo más pesada.

De manera similar, para Android, [Dalvik](#), que es la [máquina virtual](#) utilizada originalmente por Android, y lleva a cabo la transformación de la aplicación en instrucciones de máquina, que luego son ejecutadas por el [entorno de ejecución](#) nativo del dispositivo. En versiones más modernas de **Android**, **ART** ha reemplazado a **Dalvik**.

Parte II.- Entorno de trabajo. Instalación y primer programa con Android Studio.



Caso práctico



Juan y María conocen lo fundamental de los dispositivos móviles (las peculiaridades de su hardware, sus sistemas operativos) sobre los que sus aplicaciones van a ejecutarse, y juntos han tomado la decisión de trabajar con **Android Studio**, el lenguaje **Java** y los archivos **XML**.

Q Mikhail (<https://www.pexels.com/es-es/foto/empresario-hombre-persona-mujer-6592670/>) (Pexels)

- María, ¿comenzamos a descargar el software donde desarrollaremos las aplicaciones? -pregunta Juan.

- Sí -responde María-. Tengo ganas de comenzar a programar.

Ada, que les está escuchando, sabe por su experiencia que todavía les quedan cosas por hacer antes de comenzar la instalación.

- Chicos, veo que tenéis muchas ganas pero antes de comenzar a programar e instalar el **IDE** que hemos seleccionado, es imprescindible consultar los requisitos del software -les indica Ada-. Además también hay que revisar las recomendaciones de instalación del fabricante del IDE.

- De acuerdo -aceptan María y Juan al unísono-. Comenzaremos revisando los requisitos y recomendaciones para la instalación del software. ¡Muchas gracias por el consejo!



Q [Ministerio de Educación y Formación Profesional.](#)

(Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

[Aviso Legal](#)

1.- Módulos para el desarrollo de aplicaciones.

Instalación de Android Studio



Caso práctico



🔍 Mikhail (<https://www.pexels.com/es-es/foto/empresario-hombre-persona-mujer-6592558/>) (Pexels)

Juan, por iniciativa propia, ya ha avanzado en este trabajo previo a la instalación de cualquier software de uso profesional. De hecho, a partir de la información que Ada le ha transmitido en estas horas de trabajo previas al desarrollo, ya ha navegado por varias páginas de la web developer.android.com.

-Yo tengo la **descripción general** de la herramienta y sus componentes, una **guía**

de instalación y el **enlace para su descarga** con los requisitos de hardware necesarios -indica Juan-.

-¡Qué buen trabajo has realizado, Juan! -le felicita María.

-He comprobado que los requisitos son bastante altos -añade Juan-, ya que el **IDE** incluye muchas herramientas que facilitan nuestro trabajo.

-¿Es necesario instalar todas? -pregunta Pedro.

-No, en realidad no serían todas imprescindibles -responde Juan-, pero nuestro trabajo será más productivo conociéndolas e instalándolas.

De este modo, Juan ha compartido todos estos conocimientos con el equipo de trabajo, al que se ha unido Pedro. Pedro acaba de terminar otro proyecto de desarrollo web, y el cliente le ha pedido que parte de la funcionalidad desarrollada en la pagina web pueda incorporarse a una aplicación para dispositivos móviles.

Módulos para desarrollar una aplicación Android

Una vez realizado un repaso general sobre las características con las que podemos encontrarnos en el entorno de ejecución de una aplicación **Android**, ¿qué bibliotecas adicionales hay que instalar para poder desarrollar con **Android**? ¿Es suficiente con el **SDK** clásico? ¿Es necesario o aconsejable tener también un **IDE**?

Para compilar un programa con **Android** se necesita como mínimo tener instalado el **SDK** el cual se puede descargar de por separado. A partir de ahí se pueden descargar e instalar el resto de herramientas (emuladores, entornos de desarrollo, depurador, etc.).

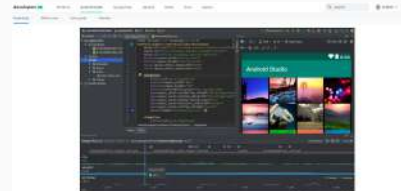
Del mismo modo que se haría con **Android**, podría hacerse con cualquier otra plataforma de desarrollo con la que se hubiera decidido trabajar: **Java ME**, **iOS**, **Windows Phone**, etc. En cada caso serían necesarias unas herramientas específicas (compiladores, editores, depuradores, emuladores, etc.). Todo esto lo

hemos tratado ya en un apartado anterior.

En el caso particular de **Android** puedes optar por la posibilidad que se ha planteado de instalar el **SDK** junto con otros elementos o bien utilizar algún otro entorno compatible que traiga todas las herramientas necesarias incorporadas. Para este curso usaremos el entorno **Android Studio**, con los paquetes necesarios para trabajar con **Android**. Precisamente en la página oficial de **Android Studio** nos da la posibilidad de instalarnos el **IDE** Android Studio, incluyendo el **SDK**, que es la forma habitual y por defecto, o bien sin incluirlo.

Entorno integrado de desarrollo: Instalación de Android Studio

Para poder desarrollar una aplicación **Android** necesitarás un entorno con el que poder realizar tu trabajo habitual de programador (editores, compiladores, depuradores, emuladores, etc.). Dependiendo de tus preferencias



Q Google Developers (License Apache 2.0)

personales y de las opciones disponibles tendrás que elegir entre las distintas alternativas que se te propongan. En nuestro caso se ha decidido que vamos a trabajar con el entorno **Android Studio**. Esta será la herramienta que utilizarás a partir de ahora y durante el resto del curso para el desarrollo de aplicaciones sobre dispositivos móviles.

Para la instalación de **Android Studio**, nos remitimos a la [página oficial de Android Studio](#).

En ella podrás visualizar un video descriptivo de todo el proceso de instalación sea cual sea tu sistema operativo, puesto que al ser una aplicación multiplataforma, está completamente testada y optimizada para los sistemas operativos más habituales (**Windows, Mac y Linux**)

En cuanto a los requisitos del sistema, necesitarás los especificados en la página oficial. Aunque será deseable disponer de los mejores recursos hardware de los que podamos disponer ya que el proceso de carga del emulador será algo tedioso por muy elevadas que sean las prestaciones del sistema.



Debes conocer

Es importante que conozcas los requisitos mínimos que Android Studio necesita para ejecutarse correctamente. Debes consultarlos en la parte inferior de la [página de descarga](#).

2.- Integración en el entorno de desarrollo.



Caso práctico

El siguiente paso es verificar que los ordenadores con los que van a trabajar cuentan con todos los requisitos necesarios para instalar el IDE **Android Studio** y descargarlo de la web oficial.

-Es importante usar fuentes oficiales de descarga para evitar problemas de seguridad en las aplicaciones que desarrollemos -indica Ada.

-Lo tendremos en cuenta -apuntan Juan, María y Pedro.

-Tras la descarga -continua Ada-, si contamos con los permisos necesarios en el sistema, procederemos a su instalación para crear nuestro primer proyecto **Android**.

-¡Qué ganas! -responde el equipo al unísono.



Q Mikhail (<https://www.pexels.com/es-es/foto/hombres-mujer-notas-sentado-6592562/>). (Pexels)



Q Rodrigo Santos (Licencia de pexels)

Una vez se ha descargado e instalado en tu ordenador la aplicación **Android Studio**, debemos ver qué pasos debemos seguir para desarrollar, probar y ejecutar una mínima aplicación móvil con esta herramienta. A lo largo de los siguientes subapartados describiremos de manera concisa cómo proceder para crear la clásica aplicación *¡Hola mundo!* que marcará nuestro punto de partida para sumergirnos en el desarrollo de aplicaciones móviles para plataforma Android. ¡Comencemos!.



Recomendación

En caso de no contar con los permisos necesarios para instalar la aplicación en tu ordenador debes conseguirlos.

Podrías plantearte usar una máquina virtual como las que has utilizado en otros módulos del curso. Pero ten en cuenta que para poder probar tus aplicaciones, Android Studio usa lo que se llama un virtualizador de dispositivos Android (AVD). Este depende de la configuración de tu SW de virtualización (VirtualBox, HyperV,...), la configuración de tu BIOS e incluso tu microprocesador y hay muchos casos en los que no se comporta de forma adecuada.

Te recomendamos por tanto que uses una máquina real para evitar problemas con los emuladores y poder incluso instalar otros diferentes a los que te ofrece Android Studio de un modo más sencillo.

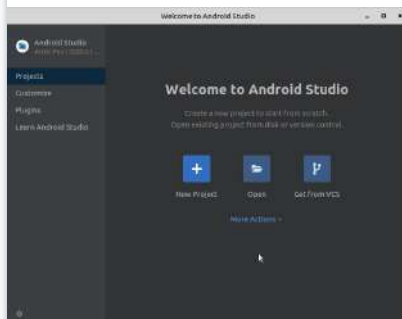
2.1.- Creación de un proyecto.

Para describir este apartado, usaremos la referencia proporcionada desde esta página de [Creación de proyectos de developer.android.com](https://developer.android.com).

Un proyecto de **Android Studio** contiene uno o más módulos que mantienen tu código organizado en unidades de funcionalidad discretas. Veremos la manera de iniciar un proyecto nuevo o importar uno existente.

Android Studio facilita la creación de apps de Android para varios factores de forma, como teléfonos y tablets, **Wear OS**, Android TV y **Android Automotive**. El asistente **New Project** te permite elegir los formatos para tu app y completar la estructura del proyecto con todo lo que necesitas para comenzar. Revisa y sigue los siguientes pasos para crear un proyecto nuevo.

Paso 1: Nuevo Proyecto



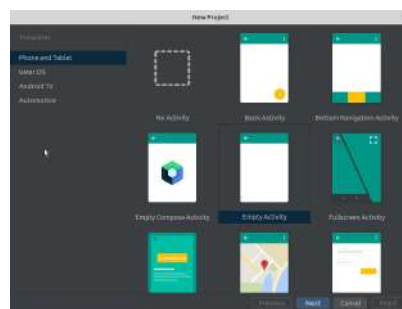
Q [Google Developers](#) (Uso educativo nc)

Si no tienes un proyecto abierto, Android Studio te muestra la pantalla de bienvenida. Para crear un proyecto nuevo, haz clic en **New Project**.

Si tienes un proyecto abierto, Android Studio muestra el entorno de desarrollo. Para crear un proyecto nuevo, haz clic en **File > New > New Project**.

Paso 2: Agregar una actividad

En la siguiente pantalla, puedes seleccionar un tipo de proyecto. Escogeremos **Phone** and **Tablet** con una actividad vacía (**Empty Activity**)



Q [Google Developers](#) (Uso educativo nc)

Paso 3: Configuración del proyecto

Introduce los siguientes datos de configuración del nuevo proyecto que estás creando:

- Nombre de la aplicación: Hola mundo

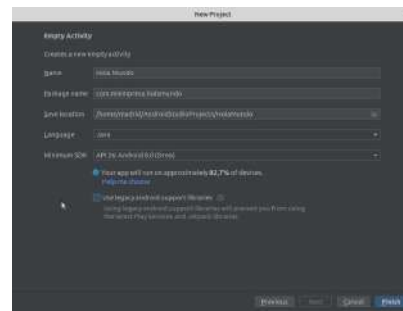
- Nombre del paquete: com.miempresa.holamundo

- Ubicación para guardar el proyecto: por defecto o la que elijas

- Language: Java

- Nivel API mínimo: escoge aquel que permita que nuestra app se ejecute en valores cercanos el 100% de los dispositivos

- No marcar la opción Use Legacy android.support libraries



Q [Google Developers](#) (Uso educativo nc)

Pulsar el botón **Finalizar**.

A continuación debe aparecer el nuevo proyecto abierto en la ventana de Android Studio.

Paso 4: Seleccionar factores de forma y el nivel de API

Este paso no lo realizaremos ahora pero lo comentamos para verlo más adelante en el curso.

Android Platform Version	API Level	Approximate Distribution
1.0	1	0.0%
2.0	2	0.0%
2.1	3	0.0%
2.2	4	0.0%
2.3	5	0.0%
2.3.3	6	0.0%
2.3.4	7	0.0%
2.3.5	8	0.0%
2.3.6	9	0.0%
2.3.7	10	0.0%
2.3.8	11	0.0%
2.3.9	12	0.0%
2.3.10	13	0.0%
2.3.11	14	0.0%
2.3.12	15	0.0%
2.3.13	16	0.0%
2.3.14	17	0.0%
2.3.15	18	0.0%
2.3.16	19	0.0%
2.3.17	20	0.0%
2.3.18	21	0.0%
2.3.19	22	0.0%
2.3.20	23	0.0%
2.3.21	24	0.0%
2.3.22	25	0.0%
2.3.23	26	0.0%
2.3.24	27	0.0%
2.3.25	28	0.0%
2.3.26	29	50.8%
2.3.27	30	24.9%

Q [Google Developers](#) (Uso educativo nc)

Podremos seleccionar los **factores de forma** admitidos por tu app (o añadirlos con **File > New > New Module**), como teléfonos y tablets, Wear OS, Android TV y Android Automotive.

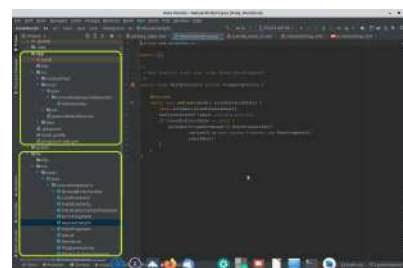
Los formatos seleccionados se convierten en los módulos de la app dentro del proyecto.

Para cada formato, también puedes

seleccionar el nivel de API para esa app. Asimismo, podremos seleccionar las versiones mínimas para los distintos factores de forma. Esta selección hará que nuestra aplicación sea compatible con más o menos versiones de Android, tal y como se muestra en esta tabla.

Por ejemplo, según la imagen, si elegimos la API29, nuestra aplicación será compatible, en principio sólo con el 50,8% de todos los dispositivos Android tablet y teléfonos que hay en uso en el mundo.

NOTA: Para eliminar módulos añadidos, nos colocamos encima de la carpeta del módulo añadido y con el botón derecho "Open Module Settings" para eliminarlo con el "-". Después suprimimos la carpeta.



Q [Google Developers](#) (Uso educativo nc)

Cada factor de forma será un módulo en nuestra app, como se puede ver en la imagen de la derecha (en este caso hemos supuesto que hemos elegido

los factores de forma Phone and Tablet y TV):

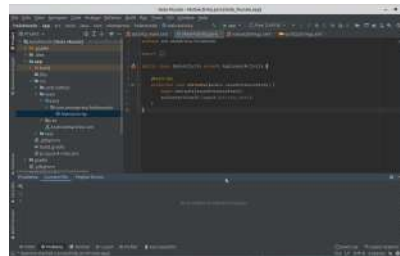
Paso 5: Actividad principal

Una **actividad** (**activity**) es un componente de la aplicación que contiene una pantalla con la que los usuarios pueden interactuar para realizar una acción, como marcar un número telefónico, tomar una foto, enviar un correo electrónico o ver un mapa. A cada actividad se le asigna una ventana en la que se puede dibujar su interfaz de usuario. La ventana generalmente abarca toda la pantalla, pero en ocasiones puede ser más pequeña que esta y quedar "flotando" encima de otras ventanas.

Una aplicación generalmente consiste en múltiples actividades vinculadas de forma flexible entre sí. Normalmente, una actividad en una aplicación se especifica como la actividad "principal" que se presenta al usuario cuando este inicia la aplicación por primera vez. Cada actividad puede a su vez iniciar otra actividad para poder realizar diferentes acciones.

Por ahora nuestro proyecto sólo tiene una actividad que es la principal representada por:

A. La clase MainActivity (con fichero de código fuente *MainActivity.java*)



Q [Google Developers](#) (Uso educativo nc)

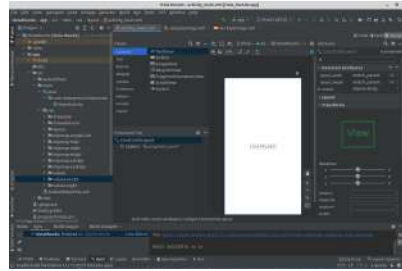
B. La disposición o layout *activity_main* (almacenada en el fichero XML: *activity_main.xml*) que contiene la información necesaria para dibujar en pantalla la interfaz gráfica de la actividad principal

Nota: no es obligatorio, sin embargo, por ahora, la primera actividad que crearemos será la actividad principal de la aplicación. Mantendremos el nombre **MainActivity** con fichero de interfaz **activity_main**, lo cual ayudará a identificar y organizar las partes de la aplicación. Por otro lado, comentar que la actividad principal no tiene por qué ser la primera actividad que se abre en la aplicación, aunque será lo más frecuente. Puede suceder, que antes de abrir la actividad principal se abra lo que se conoce como una pantalla de bienvenida (*splash screen*) que ayuda a mejorar la experiencia de usuario. Esto será habitual en aplicaciones más trabajadas o en fase de producción.

Para ver la relación existente entre la clase **MainActivity** y su disposición (**layout**) podemos pulsar Ctrl+Blzq (Botón izquierdo del ratón) sobre el nombre de la clase MainActivity en la línea de código `"public class MainActivity"` y nos llevará al fichero XML asociado. Asimismo si pulsamos Ctrl+Blzq sobre el nombre de una clase, el entorno de desarrollo Android Studio nos llevará a su definición; puedes probar pulsando sobre *Bundle* (el tipo del parámetro del método onCreate). En el caso de MainActivity no nos llevó a su definición ya que estamos en ella (en el fichero *MainActivity.java* donde se define la propia clase); en vez de eso nos mostró los sitios donde se usa esta clase (uno de ellos es el fichero XML de la disposición asociada).

Paso 6: Desarrollar tu app

Android Studio crea la estructura predeterminada de tu proyecto y se abre el entorno de desarrollo. Si tu app admite más de un formato, Android Studio crea una carpeta de módulos con archivos de origen completos para cada uno de ellos, como se muestra en la siguiente figura, donde se puede apreciar que:



Q [Google Developers](#) (Uso educativo nc)

- En la parte superior nos encontramos con un menú y/o barra de herramientas.
- En el panel lateral izquierdo tenemos un navegador de la estructura de ficheros del proyecto.
- En parte central disponemos de un panel con una paleta de elementos incorporables en una emulación de la interfaz visual de mi aplicación.
- En el panel lateral derecho disponemos de un árbol de componentes de la interfaz visual, así como un descriptor de propiedades de cada elemento incorporado.



Para saber más

Para obtener más información sobre la estructura del proyecto y los tipos de módulos de Android, lee [Información general sobre proyectos](#).

Para obtener más información sobre la manera de agregar un módulo para un dispositivo nuevo a un proyecto existente, consulta [Agregar un módulo para un dispositivo nuevo](#).

2.2.- Escritura de código.

Lo más habitual, al menos en los primeros pasos con Android Studio es partir de una aplicación previa básica que a modo de esqueleto permita agregar las particularidades necesarias para dar respuesta a las necesidades a cubrir por nuestra aplicación. Es decir, aprovecharemos las plantillas que nos proporciona Android Studio, y sobre la que elijamos, comenzaremos a trabajar para desarrollar nuestra aplicación. Concretamente, vamos a trabajar sobre la plantilla **"Empty Activity"** que tiene los elementos estrictamente necesarios para poder ejecutar la aplicación generada. Además, es habitual también, al menos, en los primeros pasos con Android Studio, mantener el nombre de actividad que pone por defecto (**MainActivity**), de forma que cuando arranque la aplicación, esta será la actividad (**activity**) que cargará en primer lugar.

Posteriormente, podremos incorporar más actividades para completar nuestra aplicación. También podremos cambiar el nombre de cualquiera de ellas, incluido **MainActivity**. Todas las actividades que vayamos incorporando en nuestra aplicación quedarán reflejadas en el fichero **AndroidManifest.xml** que veremos posteriormente. Además dicho fichero permitirá establecer cuál será la actividad con la que arrancará la aplicación.

Según la documentación proporcionada por Android Developers:

Una Activity es un componente de la aplicación que contiene una pantalla con la que los usuarios pueden interactuar para realizar una acción, como marcar un número telefónico, tomar una foto, enviar un correo electrónico o ver un mapa. A cada actividad se le asigna una ventana en la que se puede dibujar su interfaz de usuario. La ventana generalmente abarca toda la pantalla, pero en ocasiones puede ser más pequeña que esta y quedar "flotando" encima de otras ventanas.

Una aplicación generalmente consiste en múltiples actividades vinculadas de forma flexible entre sí. Normalmente, una actividad en una aplicación se especifica como la actividad "principal" que se presenta al usuario cuando este inicia la aplicación por primera vez. Cada actividad puede a su vez iniciar otra actividad para poder realizar diferentes acciones. Cada vez que se inicia una actividad nueva, se detiene la actividad anterior, pero el sistema conserva la actividad en una pila (la "pila de actividades"). Cuando se inicia una actividad nueva, se la incluye en la pila de actividades y capta el foco del usuario. La pila de actividades cumple con el mecanismo de pila "el último en entrar es el primero en salir", por lo que, cuando el usuario termina de interactuar con la actividad actual y presiona el botón Atrás, se quita de la pila (y se destruye) y se reanuda la actividad anterior.

Así pues, de forma un poco simple, podremos asociar el concepto de **Activity** al de "pantalla" o ventana de la aplicación, de forma que todas las diferentes pantallas que pudiéramos ver en una aplicación tendrán asociadas una actividad.

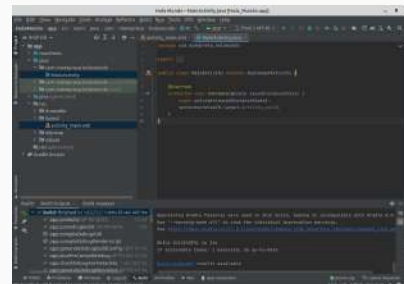
Las actividades estarán compuestas por una parte lógica y una parte gráfica.

La parte lógica se corresponderá con un archivo **.java**, que contiene la clase para poder interactuar desde esa actividad, a través de los diferentes métodos de que disponga, e incluso apoyándose en otros ficheros **.java** que contengan clases que también interactúen entre sí.

Por otro lado, la parte gráfica normalmente se corresponderá con un fichero **.xml** en el que se define la disposición (**layout**) de los elementos que se mostrarán. Así, en dicho fichero, mediante etiquetas XML específicas, se podrán incorporar todos aquellos elementos que queramos que se muestren en mi interfaz.

Veremos más adelante, cuando planteemos alguna aplicación con interfaz "programática", que será posible incorporar los elementos gráficos directamente en el fichero **.java**, pero en general, esto suele ser bastante más costoso en esfuerzo y tiempo.

Resumiendo, la escritura se hará en ficheros **.java**, para incorporar el código (utilizando lenguaje de programación **Java**) responsable de la lógica de mi aplicación y en ficheros XML para incorporar el código (utilizando lenguaje de etiquetas XML) responsable de la presentación o aspecto de la interfaz.



Q [Google Developers](#) (Uso educativo no)

Al crear nuestra primera aplicación, podemos ver ambas facetas. Para ello, completamos la creación de nuestro primer proyecto (llamado *HolaMundo*), y vemos que a través del navegador de proyectos ubicado en el marco (*frame*) de la izquierda se crean 2 ficheros relacionados entre sí:

- MainActivity, es la clase Java definida en el fichero *MainActivity.java*, y que contendrá las especificaciones lógicas de la actividad principal y de momento única, de nuestra aplicación.
- *activity_main.xml* contendrá las especificaciones de apariencia o diseño de la interfaz de esa actividad.

En los siguientes subapartados veremos el contenido de cada fichero y los explicaremos con más detalle.

2.2.1.- MainActivity.

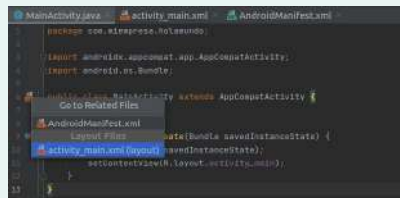


Ejercicio Resuelto. Contenido de MainActivity.

En el capítulo anterior se explicó que se iba a utilizar el lenguaje Java, el IDE Android Studio y el formato XML.

1. Averigua los archivos XML relacionados con la clase java principal de la aplicación.
2. Haz una breve explicación de cada una de las líneas de código java del archivo MainActivity.java

1. Debes pulsar en el icono "Related XML file" tal y cómo se muestra en la imagen para ver los ficheros XML que están relacionados con la clase MainActivity: *activity_main.xml* y *AndroidManifest.xml*.

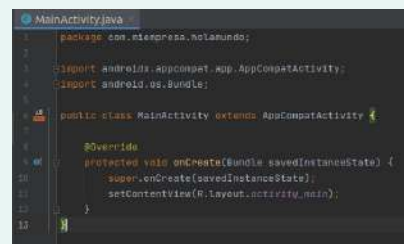


(PMDM02_CONT_R16_MainActivityRelatedXML.jpg)

 [Google Developers \(https://developer.android.com/studio\)](https://developer.android.com/studio) (Uso educativo nc)

2. Breve explicación del código:

- En la línea 1 se indica el nombre del paquete en donde se incluye nuestra clase.
- Con las líneas 3 y 4 incorporamos clases necesarias para nuestra app, las cuales están en otros paquetes.
- En la línea 6 definimos la clase llamada "MainActivity" y le aplicamos el mecanismo de herencia habitual propio de Java haciendo que extienda de la clase AppCompatActivity. Esta clase que hemos importado anteriormente dispone de las definiciones necesarias para cargar una actividad.
- En la línea 9 se incorpora una sobreescripción del método "onCreate". **Todas las actividades (activities) deben llevar el método "onCreate"**, el cual hará que inicie la actividad. Se debe pasar como parámetro un objeto de la clase *Bundle*, el cual permitirá inicializar el estado de la actividad. Si no programamos este método la actividad no se iniciará.



(PMDM02_CONT_R15_MainActivityJav

 [Google Developers \(https://developer.android.com/studio\)](https://developer.android.com/studio) (Uso educativo nc)

- ○ Lo primero que tendremos que hacer en este método será llamar al método onCreate de la clase padre pasando el objeto Bundle anterior.
- ○ A continuación, a través de la línea de código:
`setContentView(R.layout.activity_main)` conseguiremos enlazar la parte lógica con la parte gráfica. El archivo XML que va a mostrarse cuando se ejecute la clase "MainActivity" será el archivo XML llamado dentro del registro de recursos de Android Studio llamado "activity_main".

En general, cada vez que creamos una actividad nueva con un determinado nombre, debe hacer que herede de la clase AppCompatActivity (o incluso de una manera más general, de la clase Activity) y además debe tener vinculado un archivo XML que relacionaré a través del método "setContentView", pasando como parámetro el nombre que recibe dentro del registro de recursos que tiene Android y que para las actividades tiene prefijo "R.layout.", como por ejemplo sucede en el caso anterior: *R.layout.activity_main*.



Debes conocer

En la línea 3, si pulsas Ctrl+Click sobre AppCompatActivity se abre el fichero de código fuente donde se define el paquete en el que se incluye dicha clase. Idem para la clase Bundle de la sentencia import de la línea 4. Observa brevemente la descripción de esas clases en el comentario (*/** ... */*) que aparece encima de la definición de la clase.

2.2.2.- activity_main.xml.



Ejercicio Resuelto

Visualiza el contenido del archivo *activity_main.xml* que define cómo se mostrará el contenido de tu aplicación e intenta entender su contenido.

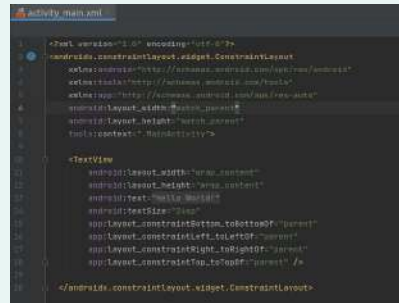
En la imagen se muestra el contenido del archivo *activity_main.xml*.

Pasemos a explicar qué significa este código XML:

- En la línea 1, ese primer elemento del fichero XML define la versión de XML y la codificación de caracteres utilizada. Habitualmente mantendremos estos valores.

- En la línea 2, la etiqueta `<android.support.constraint.ConstraintLayout...>` determina el tipo de disposición (**layout**) que queremos que se establezca, e incorpora una serie de atributos correspondientes a los espacios de nombres indicando unas URI's para identificarlos. Además tiene otros atributos que caracterizarán la anchura y altura de la disposición (**layout**) y una especificación del contexto de esa disposición (**layout**) que servirá para recordar que actividad (**activity**) la asociará.

- Por otro lado tenemos una etiqueta `<TextView...>` que corresponde a una etiqueta de tipo texto que tendrá comportamiento similar al control *Label* de Java o *label* de html. Dispondrá de atributos para especificar anchura y altura, así como el texto contenido y además una serie de atributos correspondientes a la forma en que se situará este control dentro de la disposición (**layout**).



(PMDM02_CONT_R17_activity_main_xi

Q Google Developers (<https://developer.android.com/studio>).(Uso educativo nc)

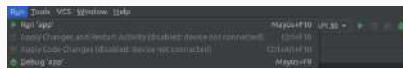


Para saber más

Para saber más sobre el acceso a recursos de Android Studio, será muy interesante el siguiente enlace: [Acceso a recursos en Android Studio](#)

2.3.- Compilación.

Para compilar el proyecto basta con ir a **Build > Make Project** y veremos que nos notifica que empezará a ejecutarse [Gradle](#) .



 [Google Developers](#) (Uso educativo no)

Para compilar y además ejecutar tu app, bastará con hacer clic en el botón **Run** situado en la barra de herramientas, o

bien, a través de menú **Run > Run 'app'**.

Android Studio compilará la app con **Gradle**, y a continuación, solicitará que seleccionemos un destino de implementación (un emulador o un dispositivo conectado) y, luego, cargará la app en dicho destino. Si no tenemos ningún dispositivo preparado para poder emular la aplicación, Android Studio mostrará un mensaje similar a este: *Error running app: No target device found.*

Podremos personalizar parte de este comportamiento predeterminado (por ejemplo, la selección de un destino de implementación automático). Veremos en siguientes apartados aspectos relacionados con el emulador y la ejecución de aplicación sobre él y otros aspectos relacionados con la ejecución en un dispositivo físico.



Debes conocer

Además, puedes ejecutar tu app en el modo de depuración. Para ello, haz clic en el botón de **Debug**. La ejecución de tu app en el modo de depuración te permite configurar puntos de interrupción en el código, examinar variables y evaluar expresiones en el tiempo de ejecución, y también ejecutar herramientas de depuración.

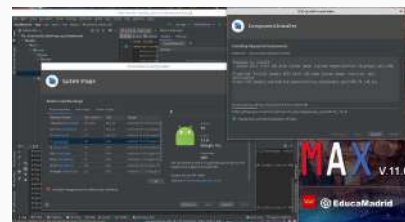
Para obtener más información, consulta [Depuración de tu app](#) .

2.4.- Emuladores.

Normalmente los entornos de desarrollo integrados suelen incorporar uno o varios emuladores que te servirán para poder probar las aplicaciones antes de instalarlas en el dispositivo para el cual se han programado. Así podrás observar si la aplicación funciona correctamente y si tiene más o menos el aspecto que tenías pensado. Hay que tener en cuenta que los emuladores no siempre van coincidir en aspecto (y a veces incluso en funcionalidad) con el dispositivo real y podrás encontrar abundantes diferencias dependiendo de diversos factores como por ejemplo el tipo de dispositivo que se emule. No es lo mismo emular un teléfono con **Windows Phone**, que un **iPhone** o un **iPad**, o un teléfono con **Android**, o un dispositivo móvil con **Java ME**.

Independientemente de la plataforma para la que vayas a desarrollar las aplicaciones (**Android**, **iOS**, **Java ME**, etc.) es importante que el entorno que elijas disponga de emuladores pues es lo que utilizarás la mayor parte del tiempo. Gran parte del proceso de depuración se realizará con el emulador, podremos ejecutar las instrucciones paso a paso, establecer puntos de ruptura (breakpoints), observar el contenido de variables, etc. Además en muchos casos es posible que no tengas a tu disposición en ese momento el dispositivo concreto para el que desees programar de manera que necesitarás el emulador.

En principio trataremos de trabajar con la aplicación de emulación proporcionada por Android (**AVD**), pero es posible utilizar otras aplicaciones gratuitas o de pago que consiguen emulaciones tan buenas o incluso más rápidas que la AVD. Éstas, normalmente se valen del uso de máquinas virtuales para poder funcionar.



Q [Google Developers](#) (Uso educativo nc)

Hay que tener en cuenta que si decidimos cambiar el tipo de emulador utilizado, la salida podrá ser ligeramente diferente en cuanto al aspecto estético del dispositivo emulado.

Si deseamos usar [Android Emulator](#)  para ejecutar nuestra app, debes preparar un **Android Virtual Device (AVD)**. Si todavía no lo hemos creado, cuando hagamos clic en Run, aparecerá un error ya que la app no encuentra un dispositivo sobre el que ejecutarse. Para solventar el problema debemos abrir **Tools > Device Manager** desde donde podremos elegir **Create Device**. Seguiremos las instrucciones del asistente **Virtual Device Configuration** para definir el tipo de dispositivo que desees emular, seleccionando la categoría (teléfono, TV, Wear, tablet o Automotive), el modelo y la imagen del sistema (a descargar).



Para saber más

Para obtener más información, consulta
[Creación y administración de dispositivos virtuales](#) .

También puedes conseguir más información sobre otros

[emuladores de Android](#), algunos orientados sólo para los juegos, pero otros, como **Genymotion** orientados al desarrollo de apps.

2.5.- Instalación y ejecución en un dispositivo real.

Como ya se ha comentado antes, algo que debes tener en cuenta cuando se desarrollan aplicaciones con la tecnología Android Studio (y en general para casi cualquier plataforma) es que el aspecto e incluso el funcionamiento que presente la aplicación puede que sea diferente en un dispositivo real con respecto al que se podía observar en un emulador. Es muy posible que su aspecto varíe según se instale en un dispositivo u otro, así como también si lo pruebas en un emulador u otro.

Esto se debe tanto a las características físicas del propio dispositivo (tamaño, resolución y colores de la pantalla, tipo de mando de juegos (joystick), botones, etc.) como a la implementación de la máquina virtual de Java que tenga ese dispositivo. Aunque la comparación no es del todo precisa, la situación podría ser similar a la que se produce cuando navegas por un mismo sitio web con ordenadores distintos, con sistemas operativos diferentes y con navegadores también distintos, obteniendo visualizaciones de la web algo diferentes.

La manera más sencilla de probar tu aplicación será transfiriendo el [archivo apk](#)  generado de la aplicación a la memoria del dispositivo donde desees realizar la prueba (a través de un cable USB, por Bluetooth, mediante una tarjeta de memoria, etc.).

Dependiendo del fabricante del dispositivo, es posible que se disponga de alguna utilidad específica del fabricante que permita realizar la instalación desde el ordenador pero también podemos disponer de utilidades no propietarias de fabricantes de móviles. Un ejemplo de este SW para Android es [adb](#). La forma de hacerlo podría cambiar dependiendo del fabricante del dispositivo así como del tipo de archivos generados (no será el mismo proceso para un iOS que para un Android, cada plataforma tendrá sus procedimientos específicos de instalación de software).

En el caso de Android, será imprescindible activar la opción de **Depuración por USB**. Puedes ver más información al respecto en este vídeo:

https://www.youtube.com/embed/Zqg6Kl46_c4

Q El Androide Libre. (Todos los derechos reservados)

Descripción textual del vídeo 



Para saber más

Para obtener más información, consulta
Ejecutar apps en un dispositivo de hardware .

3.- Gestión del entorno de ejecución.



Caso práctico

El equipo de desarrollo está entusiasmado comprobando las opciones que Android Studio proporciona para programar y testear con dispositivos virtuales e incluso reales.

-Tenemos muchas ganas de ponernos a escribir código de funciones -explica Pedro a Ada-, y empezar a diseñar el aspecto de las aplicaciones.

-Tranquilos, todo llegará -responde Ada-. Antes debéis seguir aprendiendo algunos detalles importantes relacionados con el contexto de una aplicación en un dispositivo móvil.

-¿Cuál es el siguiente paso? -pregunta María.

-Quiero estar segura de que todos conocéis el administrador de aplicaciones de Android -explica Ada-, y que sois capaces de instalar y desinstalar aplicaciones. No es complicado pero debo mostraros algunas peculiaridades en cuanto a las **fases** por las que una aplicación pasa.

-Sin embargo -prosigue Ada-, existen otros conceptos mucho más complejos relacionados con la Ingeniería del desarrollo del Software y es imprescindible saber que las aplicaciones pueden pasar por distintos **estados** mientras están en ejecución.

-Estamos preparados para aprender los conceptos de fases y estados -asegura Juan.

Gracias a algunos diagramas de la página web de Android Developers, Ada explica al equipo estos conceptos así como los eventos que lanzarán los cambios de los estados en las aplicaciones de Android.



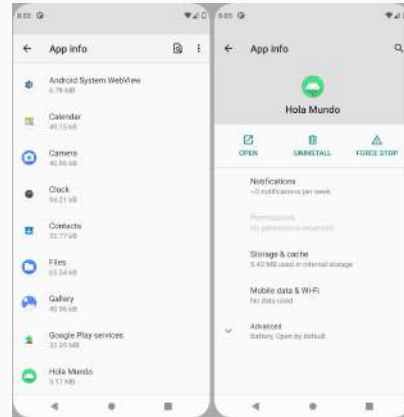
Q Mikhail (<https://www.pexels.com/es-es/foto/empresario-hombre-persona-mujer-6592371/>). (pexels)

Para describir el entorno de ejecución en un sistema como Android debemos considerar 2 aspectos importantes:

- El ciclo de vida de una aplicación. Fases y administrador de aplicaciones.
- El modelo de estados de una aplicación en ejecución.

3.1.- El ciclo de vida y el administrador de aplicaciones.

El administrador de aplicaciones o gestor de aplicaciones permite mostrar todas las apps que tenemos instaladas en nuestro dispositivo y controlar de un vistazo las que se está ejecutando, desinstalarlas o detenerlas, o incluso conocer en cada momento los recursos que está consumiendo cada una de ellas. Llevará a cabo esencialmente dos funciones:



- La gestión del ciclo de vida de las aplicaciones en el dispositivo.
- El control de los estados por los que pasa la aplicación mientras reside en la memoria del dispositivo, es decir, mientras esté en ejecución.

Q [Google Developers](#) (Uso educativo nc)

El ciclo de vida de una aplicación de Android

El ciclo de vida de una aplicación de Android pasa por cinco fases: descubrimiento, instalación, ejecución, actualización y borrado.

- **Descubrimiento (o localización).** Consiste en el proceso por el cual un usuario localiza una aplicación Android a través de su dispositivo. El gestor de aplicaciones es el que proporciona los mecanismos necesarios para realizar la elección de la aplicación que se desea descargar (mediante un cable USB, Bluetooth, conexión a Internet, etc.).
- **Instalación.** Una vez que la aplicación ha sido descargada en el dispositivo, ésta puede ser instalada. El gestor de aplicaciones de Android se encargará de controlar el proceso de instalación informando al usuario de la evolución de este proceso. Una vez instalado la aplicación, éste permanecerá en el dispositivo (en memoria persistente) hasta que el usuario decida borrarlo.
- **Ejecución.** El gestor de aplicaciones de Android es también el que permite poner en ejecución la aplicación. Una vez que una aplicación está en ejecución, el gestor de aplicaciones también tendrá el control de los distintos estados por los que pueda pasar dicha aplicación dependiendo de los eventos que se vayan desencadenando durante esa ejecución.
- **Actualización.** Una vez que una aplicación ha sido descargada, el gestor de aplicaciones de Android debe ser capaz de detectar si se trata de una actualización de una aplicación ya presente en el dispositivo. En tal caso se informaría al usuario para que decidiera si quiere llevar a cabo o no la actualización.
- **Borrado.** Si el usuario así lo desea, el gestor de aplicaciones de Android se encargará de borrar la aplicación del dispositivo. Normalmente se solicitará confirmación y se informará de las posibles incidencias durante el proceso de borrado.

3.2.- Modelo de estados de una aplicación.

De manera similar a como el planificador de procesos (scheduler) en sistemas operativos se encarga de gestionar el estado de los procesos (programas en ejecución), el Administrador de Aplicaciones es también el encargado de gestionar los estados de una aplicación durante su ejecución.

Cuando una aplicación se pone en ejecución es cargada en la memoria del dispositivo mediante un **proceso, entrando en funcionamiento una activity**. Es una diferencia con respecto a otros paradigmas que cargan la aplicación mediante el método **main()**. El ciclo de vida de cualquier activity dependerá de cómo esté relacionado con otras actividades, con sus tareas y con la pila de actividades que tenga la aplicación.

Según [Android Developer](#) , los usuarios cambian el estado de los procesos de las apps según interactúan con ellas. Los cambios en el estado del proceso van a cambiar el estado de la activity que se está ejecutando. Básicamente estarán en tres estados:

- **Reanudada:** La actividad se encuentra en el primer plano de la pantalla y tiene la atención del usuario. (A veces, este estado también se denomina "en ejecución").
- **Pausada:** Otra actividad se encuentra en el primer plano y tiene la atención del usuario, pero esta todavía está visible. Es decir, otra actividad está visible por encima de esta y esa actividad es parcialmente transparente o no cubre toda la pantalla. Una actividad pausada está completamente "viva" (el objeto Activity se conserva en la memoria, mantiene toda la información de estado y miembro y continúa anexado al administrador de ventanas), pero el sistema puede eliminarla en situaciones en que la memoria sea extremadamente baja.
- **Detenida:** La actividad está completamente opacada por otra actividad (ahora la actividad se encuentra en "segundo plano"). Una actividad detenida también permanece "viva" (el objeto [Activity](#)  se conserva en memoria, mantiene toda la información de estado y miembro, pero no está anexado al administrador de ventanas). Sin embargo, ya no está visible para el usuario y el sistema puede eliminarla cuando necesite memoria en alguna otra parte.

Si se pausa o se detiene una actividad, el sistema puede excluirla de la memoria al solicitarle que se cierre (llamando a su método **finish()**), o simplemente eliminando su proceso. Cuando se vuelve a abrir la actividad (después de haber sido eliminada o destruida), es necesario crearla nuevamente.

Cuando una actividad entra y sale de los diferentes estados que se describieron más arriba, esto se notifica a través de diferentes métodos llamados **callback**. Todos los métodos callback son enlaces que puedes invalidar para realizar las tareas correspondientes cuando cambia el estado de la actividad. La siguiente actividad de ejemplo incluye todos los métodos fundamentales del ciclo de vida.

NOTA: La implementación que realices de estos métodos del ciclo de vida siempre debe llamar a la implementación de la superclase antes de realizar cualquier otra tarea, como se muestra en los ejemplos.

Ejemplo

```
1 public class ExampleActivity extends Activity {
2
3
4     @Override
5
6
7     public void onCreate(Bundle savedInstanceState) {
8
9
10        super.onCreate(savedInstanceState);
11
12        // The activity is being created.
13
14
15    }
16
17
18    @Override
19
20
21    protected void onStart() {
22
23
24        super.onStart();
25
26
27        // The activity is about to become visible.
28
29
30    }
31
32
33    @Override
34
35
36    protected void onResume() {
37
38
39        super.onResume();
40
41
42        // The activity has become visible (it is now "resumed").
43
44
45    }
46
47
48    @Override
49
50
```

```

protected void onPause() {

    super.onPause();

    // Another activity is taking focus (this activity is about to
    // be paused)

}

@Override

protected void onStop() {

    super.onStop();

    // The activity is no longer visible (it is now "stopped")

}

@Override

protected void onDestroy() {

    super.onDestroy();

    // The activity is about to be destroyed.

}

}

```

Los 3 bucles del ciclo de vida de un activity

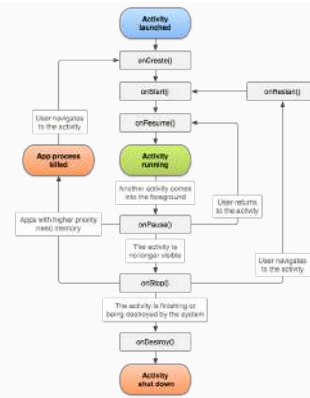
En conjunto, estos métodos anteriores definen el ciclo de vida completo de una actividad. Al implementar estos métodos, puedes monitorear los tres bucles anidados en el ciclo de vida de la actividad que puedes ver representados en la ilustración así como el camino que debe tomar una actividad entre estados. Los rectángulos representan los métodos callback que puedes implementar para realizar operaciones cuando la actividad cambie de estado.

- El ciclo de vida **completo** de una actividad transcurre entre la llamada a **onCreate()** y la llamada a **onDestroy()**. Tu actividad debe configurar el

estado "global" (como la definición del diseño) en `onCreate()`, y liberar todos los recursos restantes en `onDestroy()`. Por ejemplo, si hay un subproceso de descarga de datos de la red en ejecución en segundo plano en tu actividad, esta podría crear ese subproceso en `onCreate()` y luego detenerlo en `onDestroy()`.

- El ciclo de vida **visible** de una actividad transcurre entre la llamada a `onStart()` y la llamada a `onStop()`. Durante ese tiempo, el usuario puede ver la actividad en pantalla e interactuar con ella. Por ejemplo, se llama a `onStop()` cuando se inicia una nueva actividad y esta ya no está visible. Entre estos dos métodos, puedes conservar los recursos necesarios para mostrarle la actividad al usuario más tarde. El sistema podría llamar a `onStart()` y `onStop()` muchas veces durante el ciclo de vida completo de la actividad, ya que la actividad pasa de ser visible y a estar oculta para el usuario. No uses estos métodos para hacer acciones que sólo se realizar una vez.

- El ciclo de vida **en primer plano** de una actividad transcurre entre la llamada a `onResume()` y la llamada a `onPause()`. Durante ese tiempo, la actividad se encuentra al frente de todas las demás actividades en la pantalla y tiene el foco en la interacción del usuario. Con frecuencia, una actividad puede entrar y salir de primer plano, por ejemplo, se llama a `onPause()` cuando el dispositivo entra en suspensión o cuando aparece un diálogo. Dado que este estado puede cambiar con frecuencia, el código en estos métodos debe ser bastante liviano para evitar las transiciones lentas que hacen que el usuario deba esperar.



Q [Google Developers](#) (Uso educativo

nc)

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0 (<http://creativecommons.org/licenses/by-sa/4.0/>).